IBM Innovation Centre for Education

NCS Nihon Communication Solutions

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design on End-to-End Encryption for Supply Chain Cloud App

Student Name: T Abdul Sameer
Register Number: 23BCCE135
Course: BCA in Cloud Computing, Cyber Security & Ethical Hacking with IBM

Industry Mentor: Mr. Shashank

Guided by: Ms. Vijayalaxmi

Table of Contents

1. Introduction 1
1.1 Scope of the Document 1
1.2 Intended Audience 1
1.3 System Overview 1
2. System Design 2
2.1Application Design 2
2.2 Process Flow 2
2.3 Information Flow 3
2.4 Components Design 3
3. Data Design 5
3.1 Data Model 5
3.2 Data Storage 5
3.3 Data Access Mechanism 5
4. Interfaces 6
5. State and Session Management 7
6. Non-Functional Requirements 8
6.1 Security Aspects 8
6.2 Performance Aspects 8
6.3 Usability 8
7. References

1. Introduction

1. Scope of the Document

This document provides the Low-Level Design (LLD) of the project "End-to-End Encryption for supply chain cloud apps". It explains the internal working of the system, including modules, functions, data handling, and detailed process flow used for secure communication in supply chain cloud environments.

1.2 Intended Audience

This document is intended for developers, project evaluators, and students who need a detailed understanding of the system design and implementation.

1.3 System Overview

The system is a web-based application that ensures secure communication by encrypting and decrypting supply chain messages using the Fernet encryption technique. It simulates secure data transmission between different stakeholders such as manufacturers, suppliers, warehouses, and distributors in supply chain cloud environments.

2. System Design

1. Application Design

The application follows a **client-server model** where the user interacts with a web page and the backend processes the data.

- The **frontend** is created using HTML and CSS. It allows the user to enter messages and view the output.
- The **backend** is developed using Python Flask, which handles user requests.
- The **encryption process** uses the Fernet algorithm to secure the supply chain data.

2.2 Process Flow

1. The Sender enters supply chain data in the application.
2. The data is encrypted using a secret key (Fernet algorithm).
3. The encrypted data is stored or transmitted securely.
4. The Receiver enters the same secret key to access the data.
5. The system decrypts the data and displays the original supply chain information.

2.3 Information Flow

The information flow in the system describes how data is processed and transformed from input to output.

Input: The sender enters plain text supply chain data into the system.

Processing: The data is encrypted into ciphertext and later decrypted using the Fernet algorithm and a secret key.

Output: The system produces encrypted supply chain data for secure storage and the original data after decryption.

The system ensures that sensitive data is never stored in plain text form, thereby maintaining data confidentiality and security.

2.4 Components Design

The system is divided into different components, each performing a specific function. This modular design makes the system simple and easy to manage.

1. User Interface Component

- Developed using HTML and CSS
- Allows the sender to enter supply chain data
- Displays encrypted and decrypted output to the receiver

2. Backend Component (Flask)

- Developed using Python Flask
- Handles user requests and responses

3. Encryption Component

Encrypts the plain text message

Uses Fernet algorithm with a secret key

Converts readable text into encrypted format

4. Decryption Component

Decrypts the encrypted supply chain data

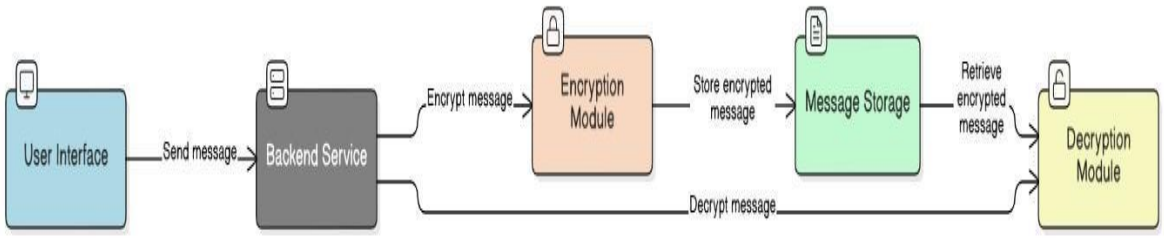
Uses the same secret key

Converts encrypted data back to original information

5. Storage Component

Stores encrypted supply chain data in a file (supply_chain_data.txt)

Ensures that only encrypted data is saved



eraser

4. Interfaces

1. User Interface

The user interface is built using HTML and CSS. It includes:

A text box to enter the message

An **Encrypt** button

A **Decrypt** button

A section to display the result

The interface is simple and easy to use.

4.2 Backend Interface

The backend is developed using Flask. It has two main routes:

/ encrpt→ Encrypts the message

/decrypt → Decrypts the message

These routes connect the frontend with the backend and handle data processing.

3. Data Design

1. Data Model

The system uses a simple data model to handle secure communication:

Plain Text supply chain data: Input provided by the sender

Encrypted supply chain data: Output generated after encryption

Secret Key: Used for both encryption and decryption

The data is converted from readable format to encrypted format to ensure security.

2. Data Storage

The system stores data using files:

Supply_chain_data.txt → Stores encrypted **supply chain data**

key.key → Stores the secret encryption key

Only encrypted data is stored to maintain confidentiality and prevent unauthorized access.

3. Data Access Mechanism

The system accesses data using file handling operations:

The key is read from the key.key file

Encrypted data is written to and read from

supply_chain_data.txt

The same key is used for both encryption and decryption This ensures secure and efficient data handling.

5. State and Session Management

The system uses role-based authentication to control access.

Login & Authentication

Users must log in with valid credentials. The system supports four roles: Admin, Manufacturer, Warehouse, and Distributor. Access is restricted based on role permissions.

Role-Based Access Control

Admin: Full access (including audit logs and key management)

Manufacturer: Access limited to sending product and shipment data

Warehouse: Access limited to receiving and updating stock status

Distributor: Access limited to dispatch and delivery information

Session Handling

Session data is stored in browser memory (localStorage) and cleared on logout.

Audit Trail

Key actions (login, logout, encryption activity, failed login attempts) are logged with timestamps. Audit logs are accessible only to Admins.

6. Non-Functional Requirements

1. Security Aspects

Implements End-to-End Encryption (E2EE)

Ensures confidentiality of supply chain data.

Prevents storage of plain text data

Uses a secret key for encryption and decryption

Provides role-based access control for secure operations

2. Performance Aspects

Fast execution of encryption and decryption operations

Minimal response time for user actions

Efficient handling of data storage and retrieval

3. Usability

Simple and user-friendly interface

Easy navigation for different user roles

Clear display of input and output messages

7. References

Python Official Documentation
Flask Framework Documentation
Cryptography Library (Fernet) Documentation